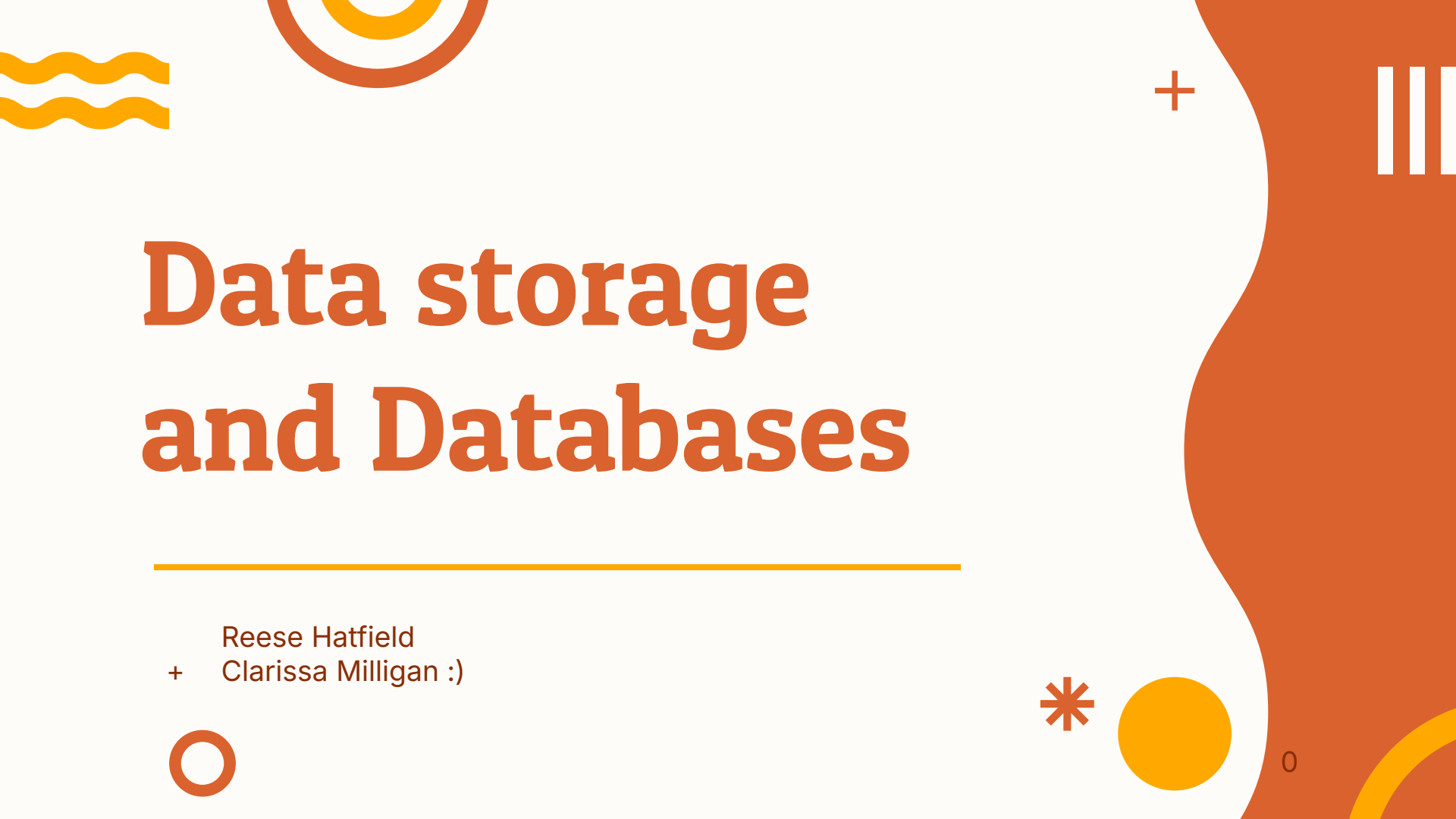
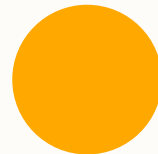




Data storage and Databases

+ Reese Hatfield
+ Clarissa Milligan :)



0



Databases

- We've talked about "data"
- How its represented
- How its used
- How we can read it
- Etc





Databases

- Plaintext data (like strings we read in) doesn't always scale appropriately
- Think back to AI models
- They need a lot of data





Databases

- Significant orders or magnitude more than anything we have seen
- Things like GPT-4 (probably) take somewhere on the scale of the entire internet to train





Databases

- Beyond machine learning, think about software companies
- Meta (Facebook + Instagram) has over 6 billion monthly activer users
- How much data per user?





Databases

- User has:
 - List of friends
 - Personalized settings
 - A specific feed
 - Username
 - Handle
 - Password
 - etc.





Databases

- How does a company store all of this data?
- At small scales, storage looks significantly different
- Think:
 - Variables
 - Files
 - etc.





Databases

- First Approach
 - Giant (likely many) file
- Store all of the data into some kind of organized file.
- Read from that file when you need to look up user data (name, password hash)





Databases

- You could format this file in many different ways
- There are some common approaches to standardize this
- But at the end of the day, we're still just storing ASCII text





Databases

- **Comma Separated Value** file
- Store one row of "headers"
- Then store each row of data

```
Username, Password, FavoriteInstructor  
Reese, p4ssw0rd, ClarissaMilligan  
Clarissa, 3ecure, TravisDoom  
...
```





Databases

- Kaggle Datasets that we saw before
- Were all in CSV format
- Big takeaway: **Tabular Data**

Username, Password, FavoriteInstructor
Reese, p4ssw0rd, ClarissaMilligan
Clarissa, 3ecure, TravisDoom
...





Databases

- Most data can be stored in such a way that it feels like a table
- But aggregating data like this can get tricky
- Need a more complex structure
- How do we store a "FriendsList"





Databases

- A user might have a list of friends
- Each friend has relevant user data
- And each friend also has a list of friends
- (This should feel like a graph from before)
- We need a way to **encode relations**





Databases

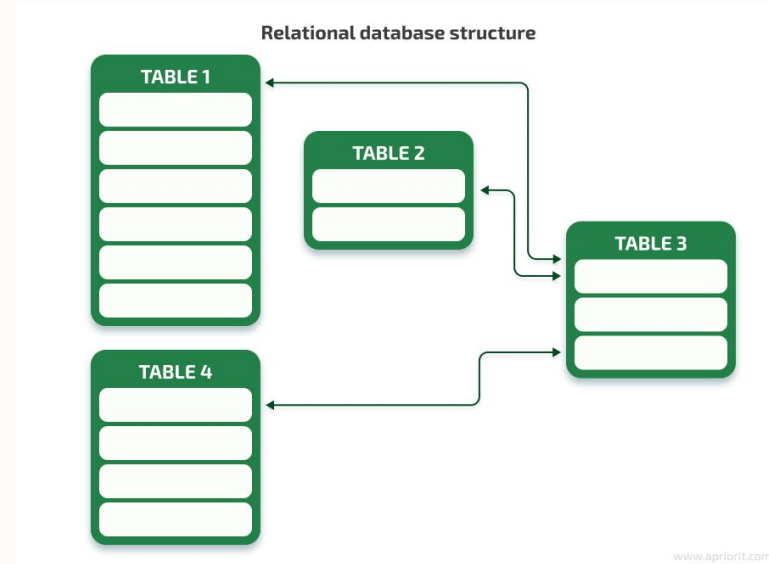
- Enter **Relational Databases**
- Encode a bunch of tables
- Also store how each table relates to other tables

- Eg. CourseEnrollment Table
- Relates to N student tables



Databases

- Relations are often stored as either
- One-to-One
- One-to-Many
- Many-to-Many*
- All entries also need a **key** to look them up with



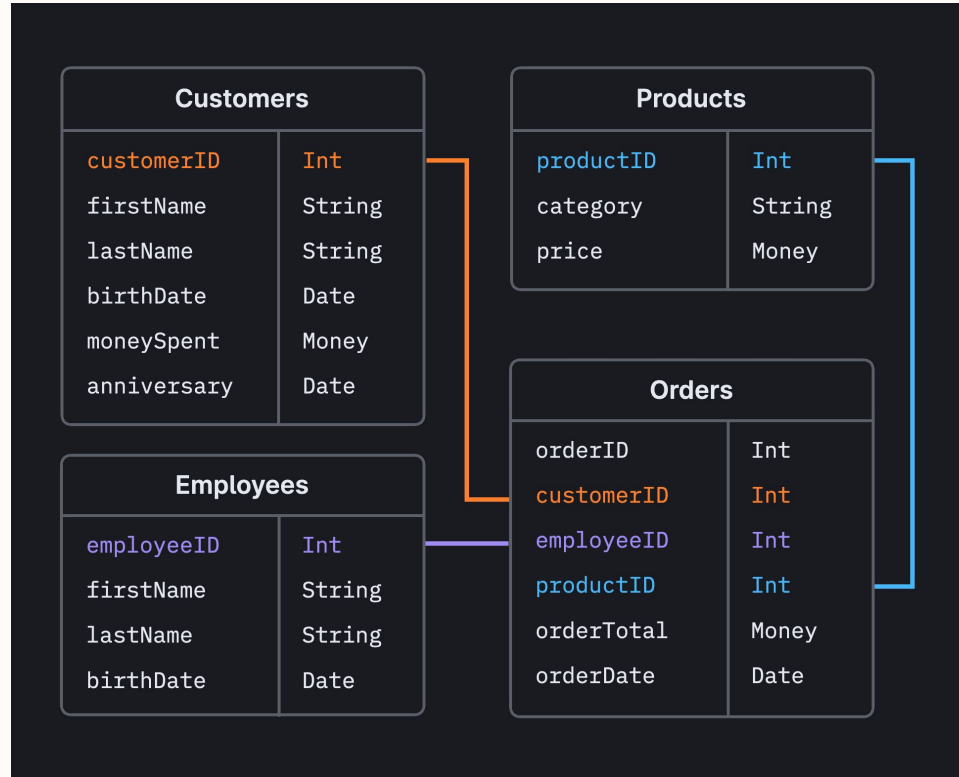


Databases

- Structure of the data is often called the **schema**
- This separates out the actual data the database is storing
- For any particular database instance
- What else could you store in a DB?
- Let's see some schema examples



Databases





Databases

- To retrieve data from a database, you need a separate “programming” language to pull data from it
- The most popular of these is called **SQL** (usually pronounced sequel*)
- Let’s see what a **SQL Query** might look like





Databases

- SELECT data from a particular table
- FROM employees

```
SELECT first_name,  
last_name, salary FROM  
employees;
```





Databases

- Grab ALL employees
- Sort them in a particular order

```
SELECT * FROM employees  
ORDER BY salary DESC;
```





Databases

- At scale, you can't always store all of this in a plaintext
- Often stored in a binary format
 - Binary in this case just means not human readable
- Often stored on the file system
- Slower than storing things in memory





Databases

- To account for this, an faster, in-memory database is often used for caching common values quickly
- Key-Value databases
- Redis is the most commonly used one
- Let's see this





Databases

- Simple
- Fast
- But very limited in scope
- Often use in combination with relational

KEY	VALUE
Reese	Blue
Clarissa	Purple
Michael	Red?
Bob	Green

